PHASE 3: SYSTEM DESIGN

1. Architecture

Selected Architectural Style: Layered Architecture

The Moderated Learning Management System (LMS) will use a Layered Architecture. The system is divided into four layers, each with a separate responsibility:

1. Presentation Layer: Provides console menus for students, lecturers, Heads of Department (HoDs) and the system administrator.

2. Business Logic Layer: Handles login permissions, course enrolment, assignment submission, deadlines, late penalties, marking, moderation and release of marks.

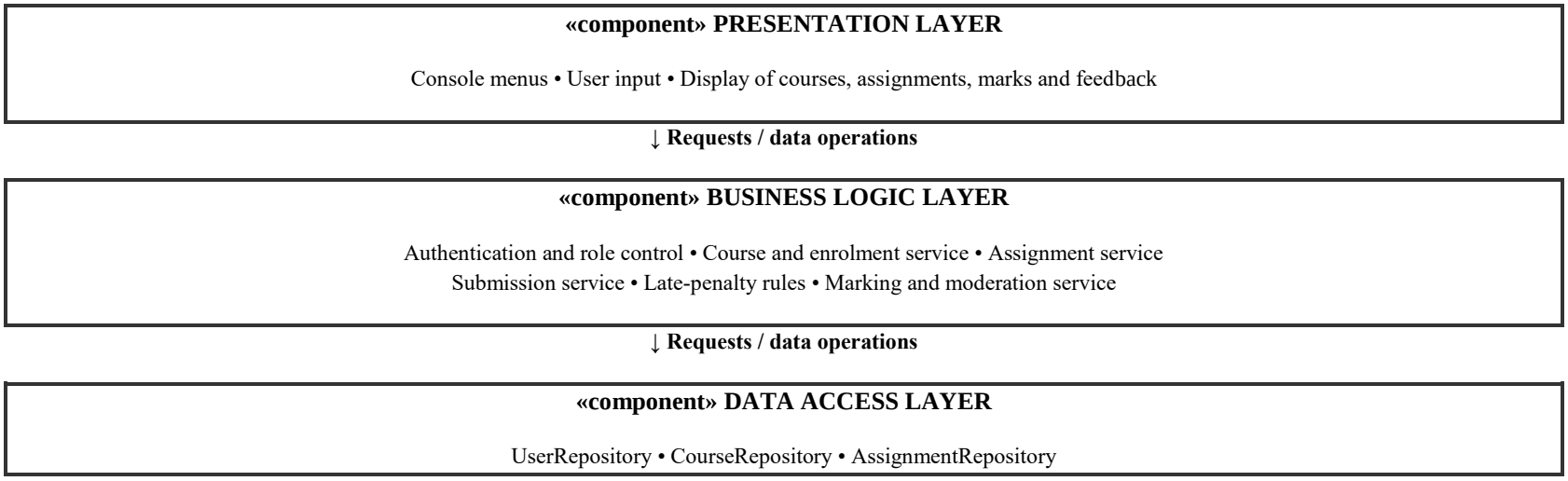
3. Data Access Layer: Saves and retrieves information through repository classes, keeping SQL code separate from business rules.

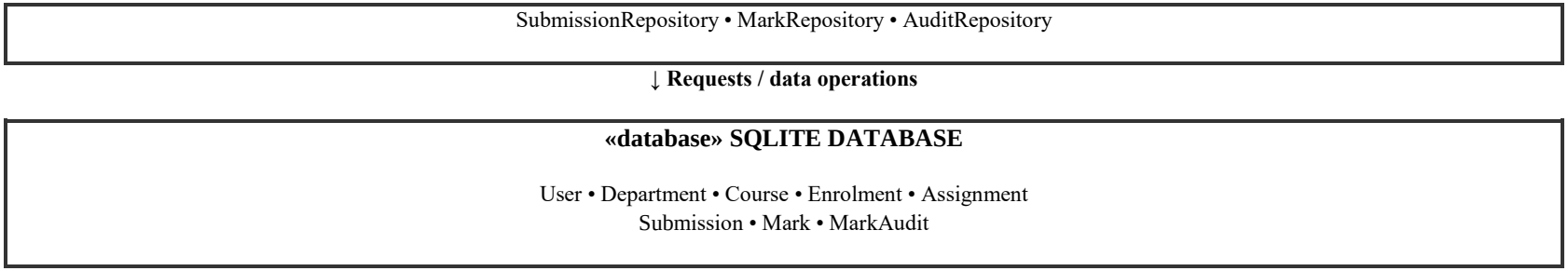
4. Database Layer: Uses SQLite to store users, departments, courses, enrolments, assignments, submissions, marks, feedback and mark-change records.

The presentation layer sends requests to the business logic layer. The business logic layer uses the data access layer, which reads from or writes to the SQLite database. Results return through the same layers to the user.

Component Diagram

The diagram shows the main components and the direction of communication between them.





Design Rationale and Rejected Alternative

Layered Architecture was selected for the following reasons:

Separation of responsibilities: Each layer has a clear purpose, making the code easier to understand and maintain.

Security: The Business Logic Layer can check a user's role before allowing access to restricted functions.

Testability: The business rules can be tested separately from the user interface and database.

Offline operation: SQLite stores data locally, so the system does not require an internet connection.

Maintainability: If the late-penalty rule changes, the business logic can be updated without redesigning the entire interface.

Simplicity: The design is suitable for a small C++17 application used by school staff who may not be IT experts.

Alternative considered and rejected: Model–View–Controller (MVC)

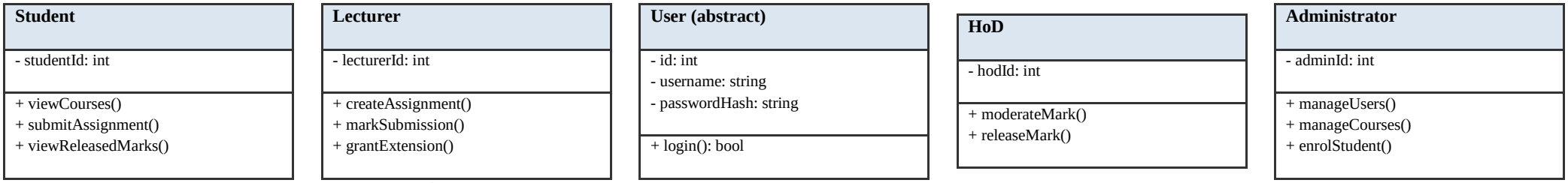
MVC separates an application into Model, View, and Controller components. It is useful for applications with complex graphical or web interfaces.

However, this project is intended to be a simple, offline C++17 application and does not require a web or mobile interface. Layered Architecture provides a clearer separation between the school's business rules and SQLite database operations.

MVC could still be used inside the Presentation Layer if the user interface becomes more complex in the future.

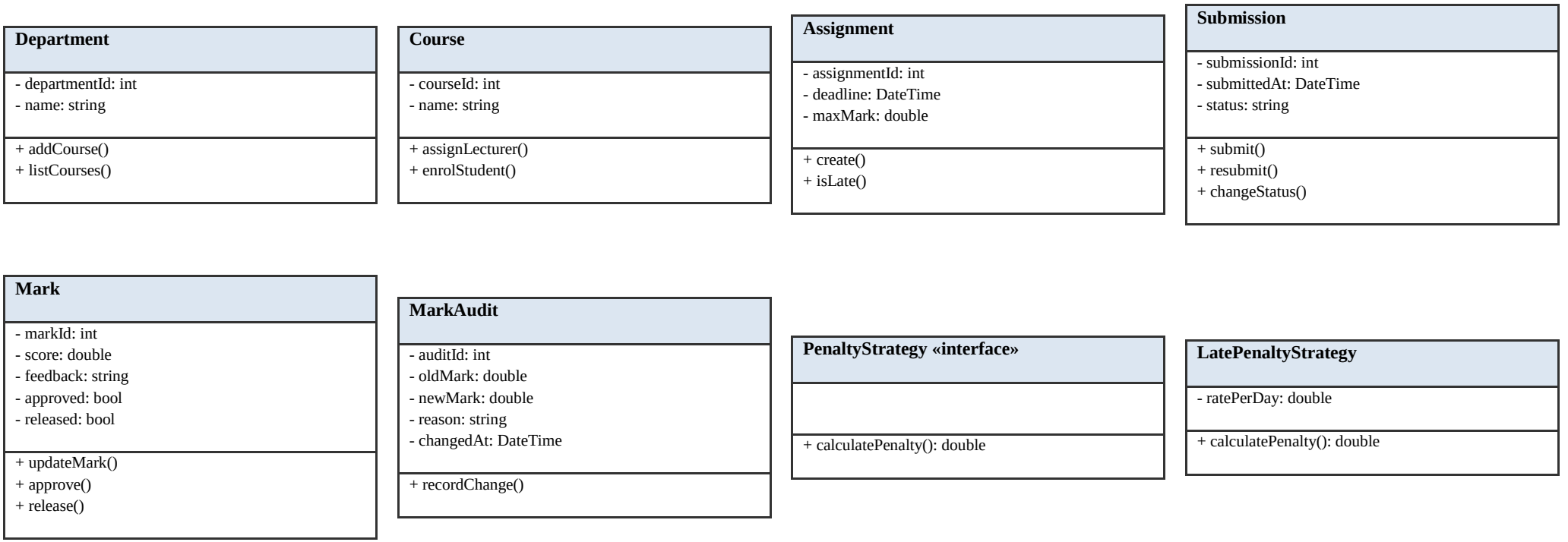
2.CLASS DIAGRAM — MODERATED LEARNING MANAGEMENT SYSTEM

1. User inheritance

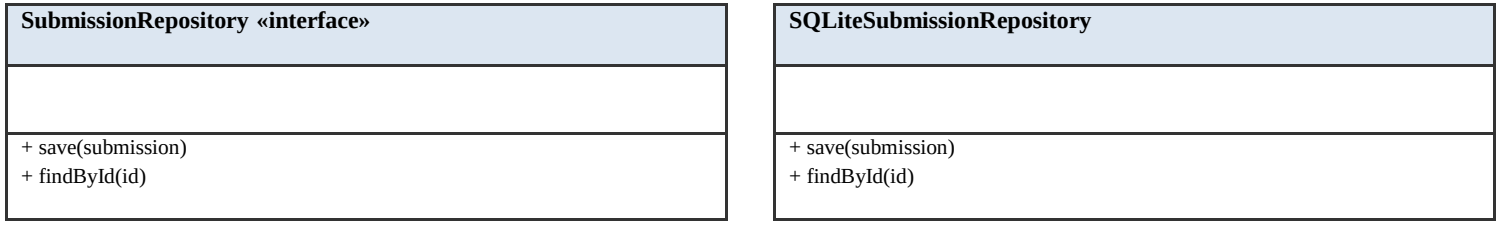


Student, Lecturer, HoD and Administrator inherit from User (generalisation).

2. Main system classes and relationships



3. Data access pattern



Relationships and multiplicities

One Department can contain many Course objects, but each course belongs to one department.

A Student can enrol in many courses, and a course can contain many students. The Enrolment class represents this many-to-many relationship.

One Course can have many assignments.

One Assignment can receive many submissions.

One student can have multiple versions of a submission for the same assignment. Each version is stored separately so resubmission history is preserved.

A submission can have no mark yet or one current mark record.

One mark can have many audit records because it may be corrected or reviewed more than once.

Student, Lecturer, HoD, and Administrator inherit common attributes and operations from User.

LatePenaltyStrategy implements the IPenaltyStrategy interface.

SQLiteSubmissionRepository implements SubmissionRepository.

3.2.5 Design patterns

Pattern 1: Strategy pattern

The Strategy pattern is used for calculating late penalties.

IPenaltyStrategy defines the penalty-calculation operation, while LatePenaltyStrategy implements the school's current rule.

The marking service uses the interface rather than placing the formula directly inside the user interface. This makes the rule easier to test and change.

The current rule is:

On-time submission or an approved extension: no late penalty.

Late by up to three days: deduct 10% of the maximum mark per late day.

More than three days late: final mark is zero.

Pattern 2: Repository pattern

The Repository pattern separates database operations from business logic.

For example, SubmissionRepository defines operations for saving and finding submissions. SQLiteSubmissionRepository implements these operations using SQLite.

This prevents SQL queries from being scattered throughout the system and makes the application easier to test and maintain.

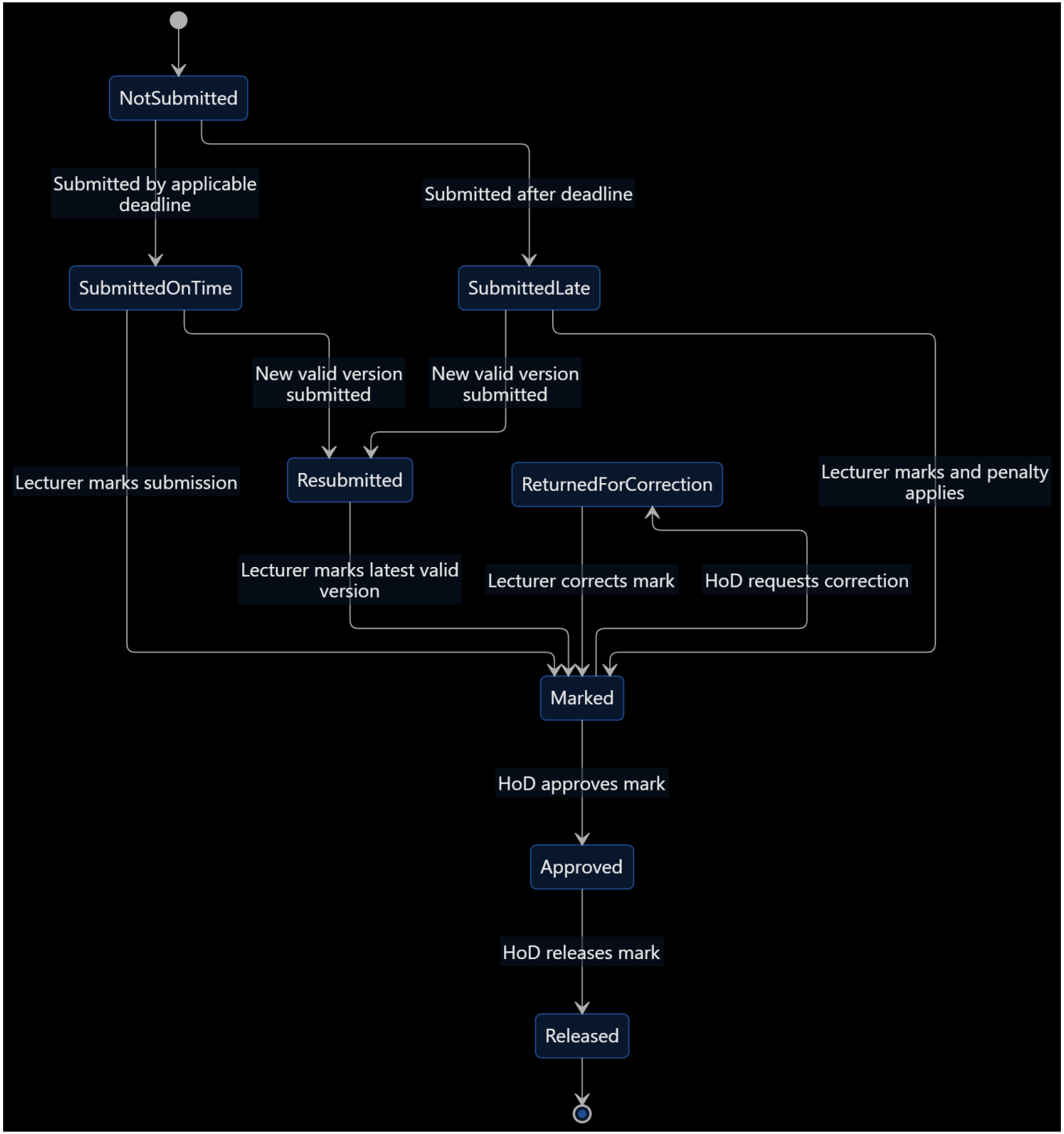
SOLID principles

Principle	Application in the LMS
S — Single Responsibility	The submission service handles submission rules, the marking service handles marking, and repositories handle data storage.
O — Open/Closed	A new penalty policy can be introduced by implementing IPenaltyStrategy, without rewriting the marking service.
L — Liskov Substitution	Each user subtype can be used where common User behaviour is expected, while role-specific permissions remain enforced.
I — Interface Segregation	Separate repository interfaces prevent classes from depending on unrelated database operations.
D — Dependency Inversion	Business services depend on interfaces rather than directly on SQLite implementation classes.

3.Behavioural models

(i) Submission state machine

The state machine shows the different states a submission passes through, from the first submission until the mark is released.

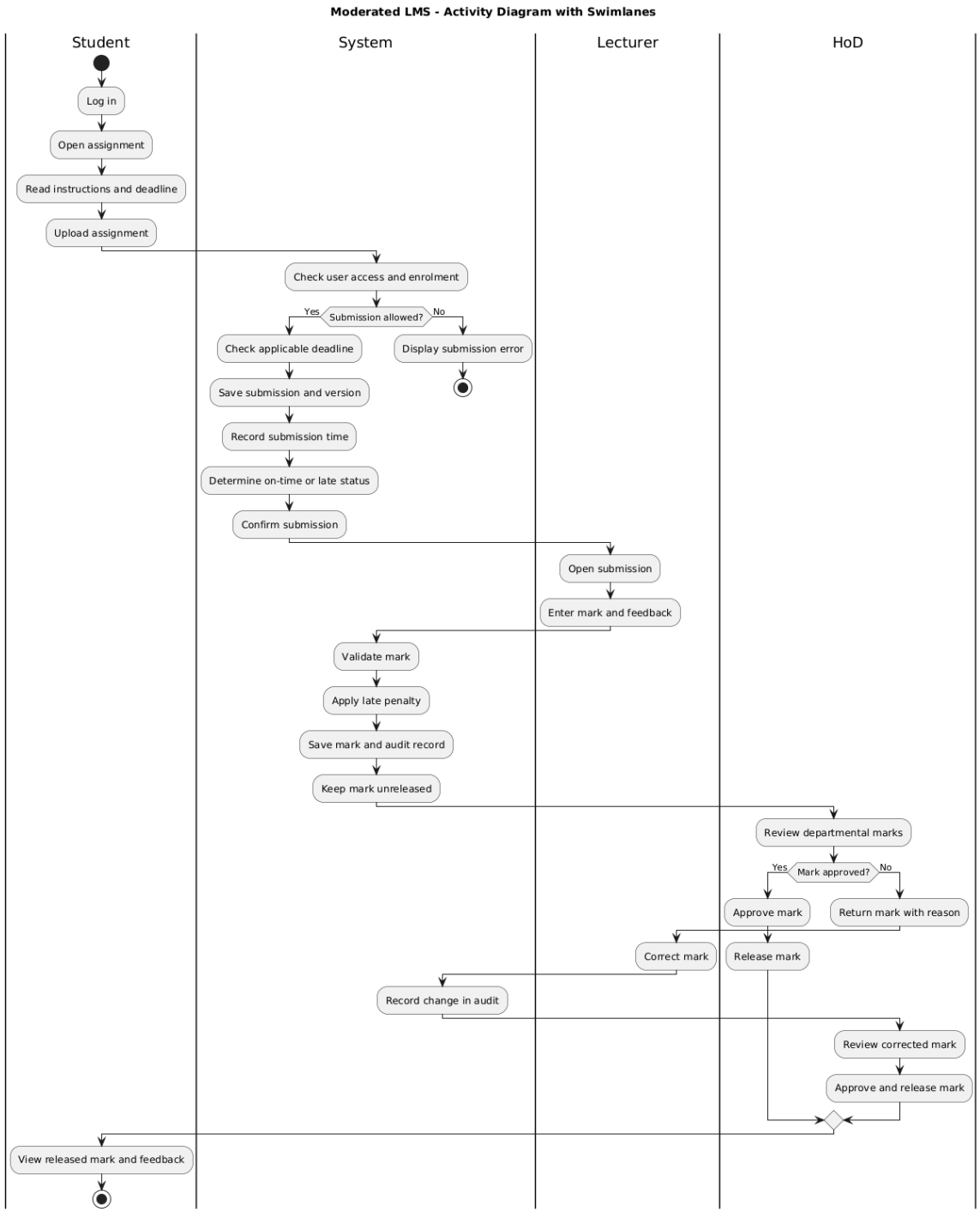

NotSubmitted
Submitted by applicable deadline
Submitted after deadline
SubmittedOnTime
SubmittedLate
New valid version submitted
New valid version submitted
Lecturer marks submission
Resubmitted
ReturnedForCorrection
Lecturer marks and penalty applies
Lecturer marks latest valid version
Lecturer corrects mark
HoD requests correction
Marked
HoD approves mark
Approved
HoD releases mark
Released

Explanation

- Not Submitted: The student has not submitted the assignment.
- Submitted On Time: The student submits by the applicable deadline, including an approved extension.
- Submitted Late: The student submits after the applicable deadline.
- Resubmitted: The student submits a new valid version. The system preserves earlier versions and identifies the latest valid version.
- Marked: The lecturer records a mark and written feedback. The system applies the late penalty when necessary.
- Returned for Correction: The HoD identifies a problem and provides a reason for returning the mark.
- Approved: The HoD approves the mark after reviewing it.
- Released: The authorised HoD releases the approved mark so the student can view it.

(ii) Activity diagram with swimlanes

This activity diagram shows the full process from submission through marking, moderation, and release. The swimlanes identify the role responsible for each activity.

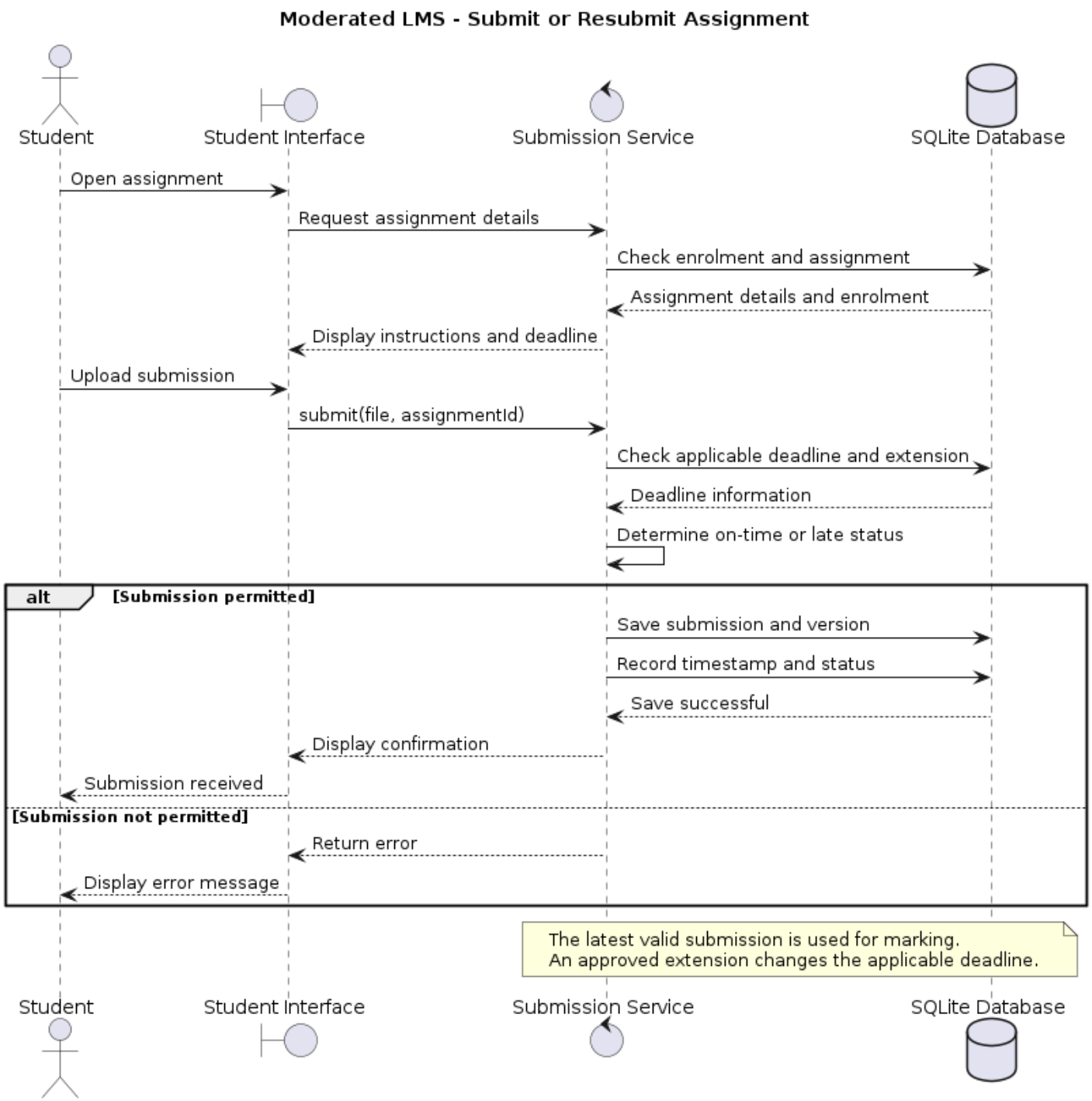

Moderated LMS - Activity Diagram with Swimlanes
Student
System
Lecturer
HoD
Log in
Open assignment
Read instructions and deadline
Upload assignment
Check user access and enrolment
Submission allowed?
Yes
No
Check applicable deadline
Display submission error
Save submission and version
Record submission time
Determine on-time or late status
Confirm submission
Open submission
Enter mark and feedback
Validate mark
Apply late penalty
Save mark and audit record
Keep mark unreleased
Review departmental marks
Mark approved?
Yes
No
Approve mark
Return mark with reason
Correct mark
Release mark
Record change in audit
Review corrected mark
Approve and release mark
View released mark and feedback

Explanation

- The system first verifies the user's access and enrolment.
- It checks the applicable deadline, including any approved extension.
- It saves the submission and records its status and version.
- The lecturer marks the submission and provides feedback.
- The system validates the mark, applies the appropriate penalty, and records the mark change.
- The HoD approves the mark or returns it for correction.
- Only an approved mark can be released.
- The student can view the mark and feedback after release.

(iii) Sequence diagram 1: Submit or resubmit assignment

This sequence diagram shows the messages exchanged when a student submits an assignment.

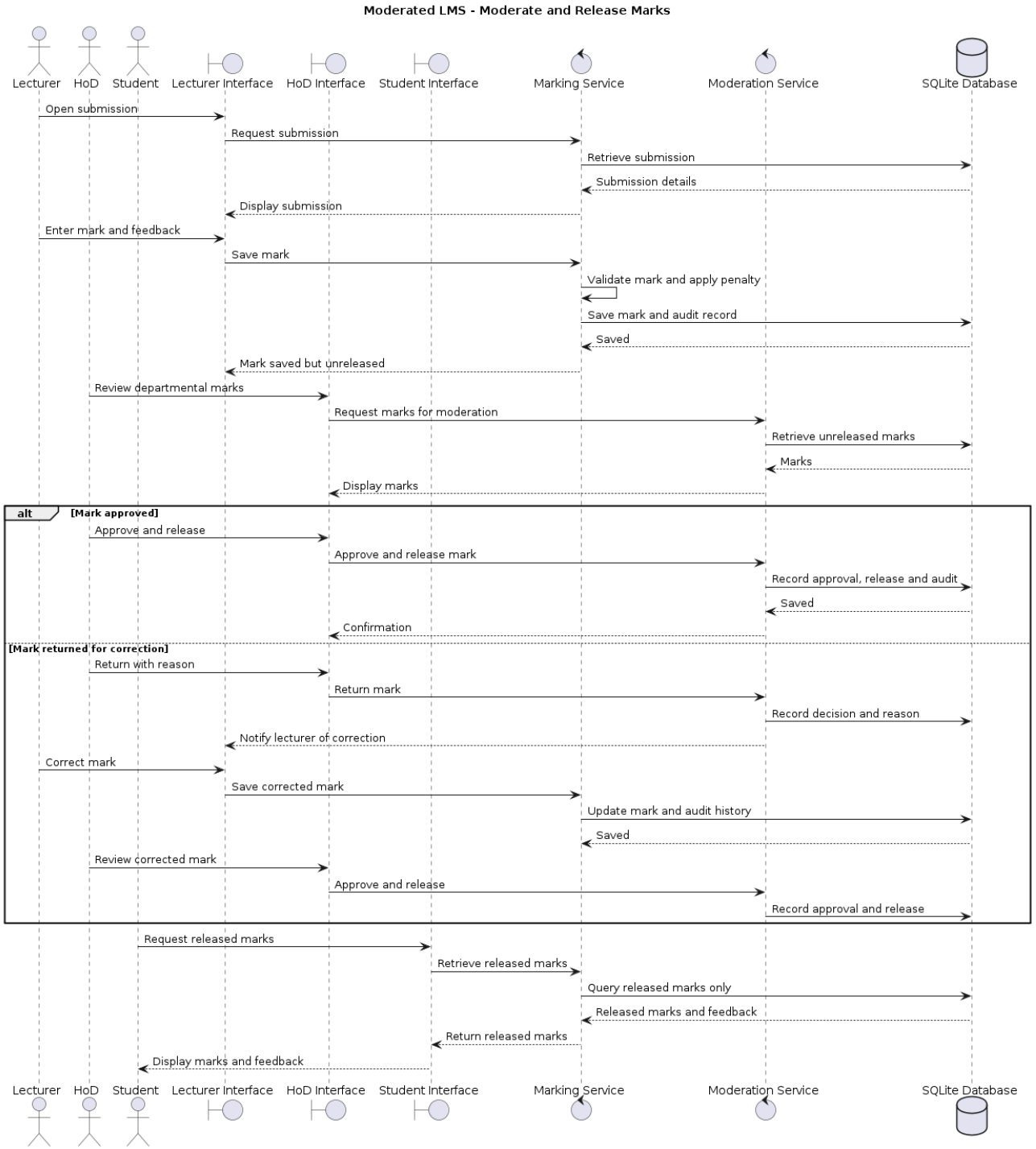

Moderated LMS - Submit or Resubmit Assignment
Student
Student Interface
Submission Service
SQLite Database
Open assignment
Request assignment details
Check enrolment and assignment
Assignment details and enrolment
Display instructions and deadline
Upload submission
submit(file, assignmentId)
Check applicable deadline and extension
Deadline information
Determine on-time or late status
alt
[Submission permitted]
Save submission and version
Record timestamp and status
Save successful
Display confirmation
Submission received
[Submission not permitted]
Return error
Display error message
The latest valid submission is used for marking.
An approved extension changes the applicable deadline.
Student
Student Interface
Submission Service
SQLite Database

Explanation

- The student selects an assignment and submits a file.
- The system checks that the student is logged in and enrolled in the course.
- The system retrieves the assignment deadline and any approved extension.
- The system checks whether this is a first submission or a resubmission.
- The submission service determines the status using the submission timestamp.
- The new version is saved, and the student receives confirmation.

Sequence diagram 2: Moderate and release marks

This sequence diagram shows how the HoD reviews marks before students can see them.


Moderated LMS - Moderate and Release Marks
Lecturer
HoD
Student
Lecturer Interface
HoD Interface
Student Interface
Marking Service
Moderation Service
SQLite Database
Open submission
Request submission
Retrieve submission
Submission details
Display submission
Enter mark and feedback
Save mark
Validate mark and apply penalty
Save mark and audit record
Saved
Mark saved but unreleased
Review departmental marks
Request marks for moderation
Retrieve unreleased marks
Marks
Display marks
alt
[Mark approved]
Approve and release
Approve and release mark
Record approval, release and audit
Saved
Confirmation
[Mark returned for correction]
Return with reason
Return mark
Record decision and reason
Notify lecturer of correction
Correct mark
Save corrected mark
Update mark and audit history
Saved
Review corrected mark
Approve and release
Record approval and release
Request released marks
Retrieve released marks
Query released marks only
Released marks and feedback
Return released marks
Display marks and feedback

Explanation

The HoD opens the marks awaiting moderation for their department. The system verifies the HoD's authority before displaying the relevant marks.

The HoD can approve a mark or return it to the lecturer with a reason. Corrections must be audited, including the previous mark, new mark, person responsible, time, and reason.

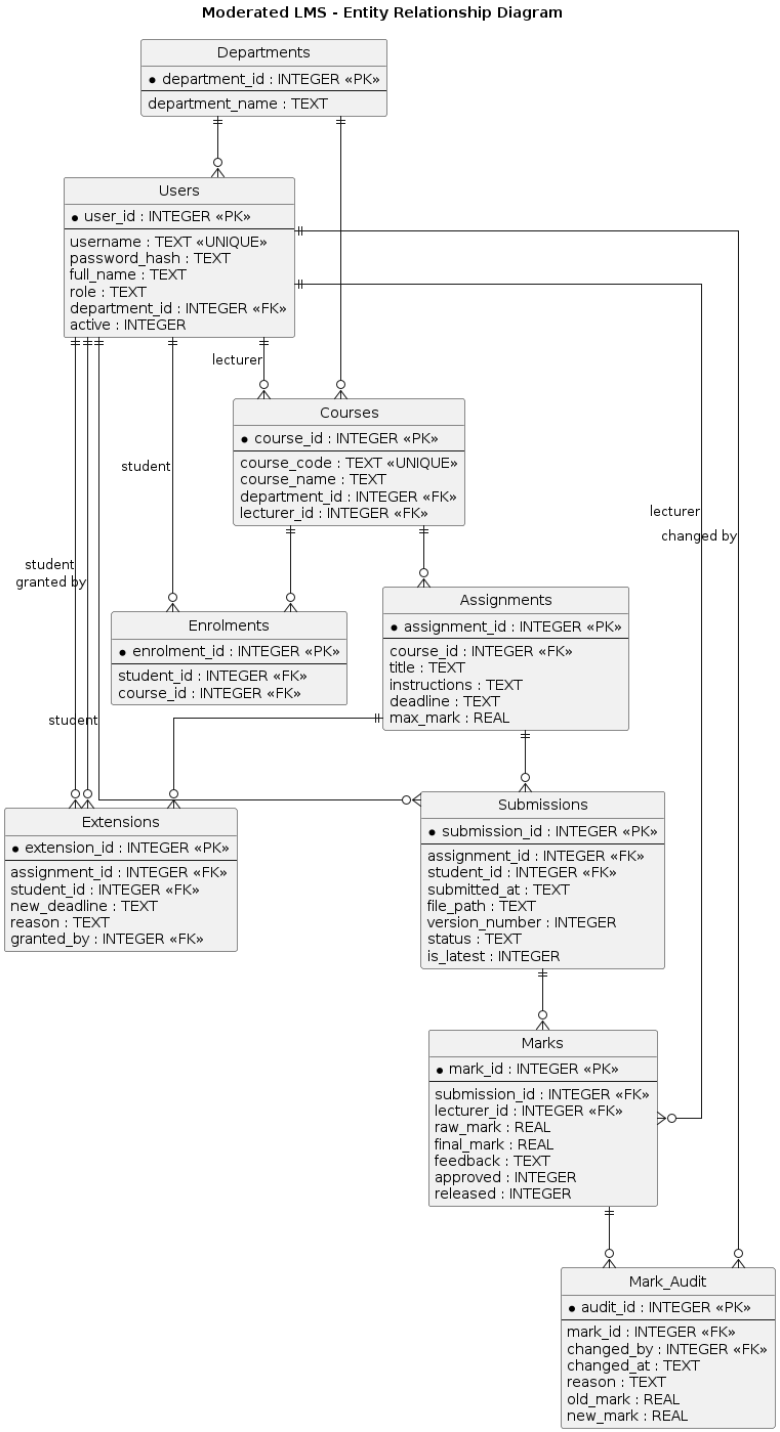
When a mark is approved, the HoD can release it. The system checks that the mark has been approved and that the HoD is authorised to release it. Students can only retrieve released marks.

4.Database Design

The database will use SQLite because the system must work offline and store information permanently. The database design supports users, departments, courses, enrolments, assignments, extensions, submissions, marks, and mark audit records.

Entity Relationship Diagram (ERD)

The ERD shows the main database entities and their relationships.


Moderated LMS - Entity Relationship Diagram
Departments
• department_id : INTEGER «PK»
department_name : TEXT
Users
• user_id : INTEGER «PK»
username : TEXT «UNIQUE»
password_hash : TEXT
full_name : TEXT
role : TEXT
department_id : INTEGER «FK»
active : INTEGER
lecturer
Courses
• course_id : INTEGER «PK»
course_code : TEXT «UNIQUE»
course_name : TEXT
department_id : INTEGER «FK»
lecturer_id : INTEGER «FK»
student
lecturer
changed by
student
granted by
Enrolments
• enrolment_id : INTEGER «PK»
student_id : INTEGER «FK»
course_id : INTEGER «FK»
Assignments
• assignment_id : INTEGER «PK»
course_id : INTEGER «FK»
title : TEXT
instructions : TEXT
deadline : TEXT
max_mark : REAL
student
Extensions
• extension_id : INTEGER «PK»
assignment_id : INTEGER «FK»
student_id : INTEGER «FK»
new_deadline : TEXT
reason : TEXT
granted_by : INTEGER «FK»
Submissions
• submission_id : INTEGER «PK»
assignment_id : INTEGER «FK»
student_id : INTEGER «FK»
submitted_at : TEXT
file_path : TEXT
version_number : INTEGER
status : TEXT
is_latest : INTEGER
Marks
• mark_id : INTEGER «PK»
submission_id : INTEGER «FK»
lecturer_id : INTEGER «FK»
raw_mark : REAL
final_mark : REAL
feedback : TEXT
approved : INTEGER
released : INTEGER
Mark_Audit
• audit_id : INTEGER «PK»
mark_id : INTEGER «FK»
changed_by : INTEGER «FK»
changed_at : TEXT
reason : TEXT
old_mark : REAL
new_mark : REAL

Explanation of relationships

A department can contain many courses.

A student can enrol in many courses, and a course can have many students. ENROLMENTS resolves this many-to-many relationship.

A course can contain many assignments.

An assignment can have many extensions and submission versions.

An extension identifies the student receiving it, the lecturer granting it, the new deadline, and the reason.

Each submission version belongs to one student and one assignment.

A submission can have zero or one current mark record.

A mark can have multiple audit records.

User foreign keys identify the people responsible for teaching, granting extensions, marking, and changing marks.

Data dictionary

The data dictionary describes the important fields in the database.

Table	Field	Type	Description
Users	user_id	INTEGER	Primary key identifying a user.
Users	username	TEXT	Unique login name.
Users	password_hash	TEXT	Secure password hash, never a plain-text password.
Users	role	TEXT	User role: STUDENT, LECTURER, HOD, or ADMIN.
Departments	department_id	INTEGER	Primary key identifying a department.
Departments	hod_user_id	INTEGER	Foreign key identifying the HoD.
Courses	course_id	INTEGER	Primary key identifying a course.
Courses	department_id	INTEGER	Department offering the course.
Courses	lecturer_user_id	INTEGER	Lecturer assigned to the course.
Enrolments	student_user_id	INTEGER	Enrolled student's user ID.
Enrolments	course_id	INTEGER	ID of the enrolled course.
Assignments	deadline	TEXT	Applicable deadline stored in a consistent date-time format.
Assignments	max_mark	REAL	Maximum possible mark.
Extensions	new_deadline	TEXT	Approved extended deadline.
Extensions	granted_by	INTEGER	Lecturer who granted the extension.
Submissions	submitted_at	TEXT	Date and time the work was submitted.
Submissions	status	TEXT	ON_TIME, LATE, or BEYOND_ALLOWED_PERIOD.

Submissions	version_no	INTEGER	Submission version number.
Marks	raw_mark	REAL	Mark before the late penalty.
Marks	final_mark	REAL	Mark after applying the penalty.
Marks	moderation_status	TEXT	PENDING, RETURNED, APPROVED, or RELEASED.
Marks	released_at	TEXT	Date and time the mark was released.
Mark_Audit	changed_by	INTEGER	User responsible for the change.
Mark_Audit	changed_at	TEXT	Date and time of the change.
Mark_Audit	old_mark / new_mark	REAL	Previous and updated marks.
Mark_Audit	reason	TEXT	Reason for the change or moderation decision.

SQLite database schema

The following SQL defines the main tables, primary keys, foreign keys, uniqueness rules, and constraints. It can be used as the starting point for implementing the database.

Below is the SQL plain text code:

```
PRAGMA foreign_keys = ON;


-- 1. Users and login details
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL
        CHECK (role IN ('STUDENT', 'LECTURER', 'HOD', 'ADMIN')),
    active INTEGER NOT NULL DEFAULT 1
        CHECK (active IN (0, 1))
);
```

```
-- 2. Departments
CREATE TABLE Departments (
    department_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    hod_user_id INTEGER,
    FOREIGN KEY (hod_user_id)
        REFERENCES Users(user_id)
);

-- 3. Courses
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    department_id INTEGER NOT NULL,
    course_code TEXT NOT NULL UNIQUE,
    course_name TEXT NOT NULL,
    lecturer_user_id INTEGER,
    FOREIGN KEY (department_id)
        REFERENCES Departments(department_id),
    FOREIGN KEY (lecturer_user_id)
        REFERENCES Users(user_id)
);
```

```
-- 4. Student enrolments
CREATE TABLE Enrolments (
    enrolment_id INTEGER PRIMARY KEY,
    student_user_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    status TEXT NOT NULL DEFAULT 'ACTIVE'
        CHECK (status IN ('ACTIVE', 'INACTIVE')),
    UNIQUE (student_user_id, course_id),
    FOREIGN KEY (student_user_id)
        REFERENCES Users(user_id),
    FOREIGN KEY (course_id)
        REFERENCES Courses(course_id)
);

-- 5. Assignments
CREATE TABLE Assignments (
    assignment_id INTEGER PRIMARY KEY,
    course_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    instructions TEXT,
    deadline TEXT NOT NULL,
```

```
    max_mark REAL NOT NULL CHECK (max_mark > 0),
    FOREIGN KEY (course_id)
        REFERENCES Courses(course_id)
);

-- 6. Individual extensions
CREATE TABLE Extensions (
    extension_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
    student_user_id INTEGER NOT NULL,
    granted_by INTEGER NOT NULL,
    new_deadline TEXT NOT NULL,
    reason TEXT NOT NULL,
    granted_at TEXT NOT NULL,
    FOREIGN KEY (assignment_id)
        REFERENCES Assignments(assignment_id),
    FOREIGN KEY (student_user_id)
        REFERENCES Users(user_id),
    FOREIGN KEY (granted_by)
        REFERENCES Users(user_id)
);
```

```
-- 7. Submission versions
CREATE TABLE Submissions (
    submission_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL,
    student_user_id INTEGER NOT NULL,
    submitted_at TEXT NOT NULL,
    status TEXT NOT NULL
        CHECK (status IN (
            'ON_TIME',
            'LATE',
            'BEYOND_ALLOWED_PERIOD'
        )),
    version_no INTEGER NOT NULL CHECK (version_no > 0),
    file_path TEXT NOT NULL,
    is_latest INTEGER NOT NULL DEFAULT 1
        CHECK (is_latest IN (0, 1)),
    UNIQUE (assignment_id, student_user_id, version_no),
    FOREIGN KEY (assignment_id)
        REFERENCES Assignments(assignment_id),
    FOREIGN KEY (student_user_id)
        REFERENCES Users(user_id)
);
```

```
-- Only one version may be marked as the latest
-- for a particular student and assignment.
CREATE UNIQUE INDEX one_latest_submission
ON Submissions(assignment_id, student_user_id)
WHERE is_latest = 1;

-- 8. Marks and feedback
CREATE TABLE Marks (
    mark_id INTEGER PRIMARY KEY,
    submission_id INTEGER NOT NULL UNIQUE,
    lecturer_user_id INTEGER NOT NULL,
    raw_mark REAL NOT NULL CHECK (raw_mark >= 0),
    final_mark REAL NOT NULL CHECK (final_mark >= 0),
    feedback TEXT,
    moderation_status TEXT NOT NULL DEFAULT 'PENDING'
        CHECK (moderation_status IN (
            'PENDING', 'RETURNED', 'APPROVED', 'RELEASED'
        )),
    released_at TEXT,
    FOREIGN KEY (submission_id)
        REFERENCES Submissions(submission_id),
```

```
    FOREIGN KEY (lecturer_user_id)
        REFERENCES Users(user_id)
);


-- 9. Mark changes and moderation audit records
CREATE TABLE Mark_Audit (
    audit_id INTEGER PRIMARY KEY,
    mark_id INTEGER NOT NULL,
    changed_by INTEGER NOT NULL,
    changed_at TEXT NOT NULL,
    old_mark REAL,
    new_mark REAL,
    reason TEXT NOT NULL,
    action_type TEXT NOT NULL,
    FOREIGN KEY (mark_id)
        REFERENCES Marks(mark_id),
    FOREIGN KEY (changed_by)
        REFERENCES Users(user_id)
);
```